was blocked because the local file is considered to be on a separate origin to the API and so browsers will block the request by default.

By default, browsers allow JavaScript to send HTTP requests only to a server on the same *origin* that the script was loaded from. This is known as the *same-origin policy* (SOP) and is an important cornerstone of web browser security. To the browser, a file URL and an HTTPS URL are always on different origins, so it will block the request. In the next chapter, you will see how to fix this with cross-origin resource sharing (CORS), but for now let's get Spark to serve the UI from the same origin as the Natter API.

> **DEFINITION** The ***origin*** of a URL is the combination of the protocol, host, and port components of the URL. If no port is specified in the URL, then a default port is used for the protocol. For HTTP the default port is 80, while for HTTPS it is 443. For example, the origin of the URL https://www.google.com/search has protocol = https, host = www.google.com, and port = 443. Two URLs have the same origin if the protocol, host, and port all exactly match each other.

The same-origin policy

The same-origin policy (SOP) is applied by web browsers to decide whether to allow a page or script loaded from one origin to interact with other resources. It applies when other resources are embedded within a page, such as by HTML `<img>` or `<script>` tags, and when network requests are made through form submissions or by JavaScript. Requests to the same origin are always allowed, but requests to a different origin, known as *cross-origin requests*, are often blocked based on the policy. The SOP can be surprising and confusing at times, but it is a critical part of web security so worth getting familiar with as an API developer. Many browser APIs available to JavaScript are also restricted by origin, such as access to the HTML document itself (via the document object model, or DOM), local data